Doctor Tracker – Project Specification

1. Project Overview

Doctor Tracker is a secure administrative web application that allows authenticated users to manage doctors and their corresponding patients. The system focuses on performance optimization, clean UX, and data visualization.

2. Core Features

2.1 Authentication

- Only authenticated users can access the portal.
- Implement secure login functionality.
- Protect all routes using proper authentication and authorization mechanisms.

2.2 Doctor Management

- Admin can:
    - Create a doctor with the following information:
        - Name
        - specialization
        - hospital

        - Phone
        - Email
    - View a list of all doctors.
    - Search doctors.
    - Filter doctors (date-wise and other relevant filters).
    - Paginate doctor records.
    - View corresponding patients for each doctor.
    - Add new patients under a specific doctor.
    - Delete patients from the doctor’s patient list.

2.3 Patient Management

- Dedicated Patient Page:
    - List all patients.
    - Edit patient information.
    - Delete patients.
    - Implement:
        - Pagination

        - Search functionality
        - Filtering (date-wise, patient-condition and other relevant filters)

3. Dashboard & Data Visualization

- Admin Dashboard should include:
    - Visual representation of data.
    - Analytics such as:
        - Total doctors
        - Total patients
        - Patients per doctor
        - Date-based statistics
    - Charts and graphs (any suitable analytics library may be used).
- Focus on:
    - Clean data aggregation
    - Optimized query performance
    - Meaningful visual insights

4. UI/UX Requirements

- The UI must be modern and visually appealing.
- Demonstrate strong frontend skills.
- Ensure:
    - Clean layout
    - Proper spacing and hierarchy
    - Responsive design
    - Smooth navigation
    - Good user experience (UX best practices)
- Clear navigation between:
    - Dashboard
    - Doctors
    - Patients

5. Performance & Optimization

- Optimize database queries for:
    - Searching

- Filtering
- Pagination

- Implement best practices:
  - Proper indexing in MongoDB
  - Efficient query patterns
  - Avoid unnecessary re-renders
  - Clean folder structure
  - Reusable components
  - API validation and error handling

6. Tech Stack

The project must be implemented using:

- **Frontend & Backend Framework:** Next.js (Full-stack inside Next.js application)
  Frontend: Next.js (separate client application)
  Backend: Node.js, Express (separate standalone server)
- **Database:** MongoDB
  Database: MongoDB
- **Backend Technologies:** Node.js, Express (integrated within Next.js architecture)
  API: RESTful API design principles (frontend

communicates with backend over REST endpoints)
- RESTful API design principles
- Proper state management and data fetching strategies

7. Evaluation Criteria

The project will be evaluated based on:

- Code structure and cleanliness
- Query optimization
- UX/UI quality
- Data visualization implementation
- Performance efficiency
- Authentication implementation
- Use of best practices in Next.js and MongoDB
- Scalability considerations

README Requirements (Standardized Template)

The README.md is the primary technical documentation. It must include:

Section	Requirement

Description	A one-paragraph "Elevator Pitch" for the app's purpose.
Setup Guide	Step-by-step local installation. Must include an .env.example file.
System Architecture	A high-level overview of the data flow and service interactions.
Technical Decisions	Deep dive into 2 specific decisions (e.g., "Why we chose Redux over Context API").
Visual Evidence	High-quality screenshots of the UI (Desktop & Mobile views).

Submit your project link and github repo with credentials if required.